

DCPO: DYNAMIC CLIPPING POLICY OPTIMIZATION

Anonymous authors
Anonymous submission

ABSTRACT

Reinforcement Learning from Verifiable Rewards (RLVR) provides an effective optimization framework for improving the reasoning behavior of large language models using outcome-level supervision. Existing policy-optimization methods such as GRPO, however, frequently produce zero-gradient updates. This failure mode is induced primarily by fixed clipping intervals for token-level probability ratios and by step-local standardization when all sampled rewards are identical, which together suppress informative gradients and reduce the effective utilization of generated responses. We propose **Dynamic Clipping Policy Optimization (DCPO)**, a policy-optimization procedure that combines two mechanisms: a dynamic clipping rule that sets token-specific clipping bounds as a function of prior token probabilities, thereby expanding admissible token-level exploration for low-probability events; and a smooth advantage standardization rule that computes reward normalization across cumulative training steps, thereby increasing response-level utilization of generated samples. Across four benchmarks and four model backbones, DCPO attains state-of-the-art performance. On Qwen2.5-Math-7B, DCPO obtains an Avg@1 of 46.7 under greedy decoding and an Avg@32 of 38.8 under 32-sample decoding on AIME24, exceeding DAPO (36.7/31.6), GRPO (36.7/32.1), and GSPO (40.0/34.9). On Qwen2.5-72B, DCPO reaches (23.3/19.0) on AIME25, outperforming GRPO (13.3/10.5), DAPO (20.0/15.3), and GSPO (16.7/9.9). In addition, DCPO increases the nonzero-advantage ratio by an average of 28% relative to GRPO across four models, doubles training efficiency relative to DAPO, and lowers the token clipping ratio by an order of magnitude compared with GRPO and DAPO while maintaining stronger task performance. These results indicate that DCPO uses generated trajectories more efficiently during RLVR-based language-model optimization.

1 INTRODUCTION

The reasoning capacity of large language models (LLMs) has become a central object of study in recent work (OpenAI, 2025; Guo et al., 2025; Comanici et al., 2025). Reinforcement Learning from Verifiable Rewards (RLVR) is a particularly relevant optimization paradigm for this setting because it uses rule-defined outcome rewards as supervision signals for policy improvement. By assigning rewards from externally verifiable correctness criteria, RLVR can improve LLM performance in domains that require multi-step deliberation and reflective decision making, including mathematics and coding (DeepSeek-AI et al., 2024).

The GRPO family of algorithms (DeepSeek-AI et al., 2024; Liu et al., 2025; Yuan et al., 2025; Zheng et al., 2025) was introduced to instantiate RLVR without an explicit critic. Although GRPO has been empirically validated in multiple studies, subsequent analyses, including DAPO (Yu et al., 2025), identify several structural limitations. In particular, GRPO can exhibit entropy collapse, in which advantage estimates decay to zero in later optimization stages. In addition, fixed or poorly calibrated probability-ratio clipping can remove learning signals associated with rarely sampled tokens, thereby restricting the diversity of admissible policy exploration.

DAPO addresses these issues by introducing Clip-Higher, which relaxes the upper clipping bound to mitigate entropy collapse, and Dynamic Sampling, which filters samples with uniform rewards so that each retained batch contains nonzero-gradient examples. These interventions nevertheless leave two limitations unresolved. (i) **Response-Level Inefficiency**: Dynamic sampling reduces sampling efficiency because generated responses with identical rewards are discarded, which slows

convergence for a fixed generation budget. (ii) **Token Clipping Deficit**: Our analysis shows that the appropriate clipping interval is token dependent; consequently, a single fixed upper bound is not sufficient to characterize the desired update constraint. Although alternative methods may avoid directly discarding zero-gradient responses, these efficiency and clipping limitations remain common.

We introduce **Dynamic Clipping Policy Optimization (DCPO)** to address these limitations through a set of probability- and response-aware modifications to policy optimization. First, DCPO uses a **smoothing technique that standardizes advantages across cumulative steps**; this procedure avoids eliminating gradients solely because the current group of responses receives identical rewards. The method combines the reward statistics of all previously generated responses for a prompt with the statistics of the current generation step, thereby supporting more efficient response-level exploration (see Section 2). Second, DCPO uses a **dynamic clipping method that adaptively adjusts the clipping bounds based on the old token-specific probabilities** rather than imposing fixed lower and upper bounds. Tokens assigned lower prior probability receive wider admissible ratio intervals, which increases token-level exploration capacity (see Section 2 and Appendix A.6). Finally, DCPO computes the mean loss over tokens within each individual response rather than averaging over the full batch, preserving the relative advantage ordering among responses to the same prompt and avoiding batch-level dilution of response importance.

For training, we merge Math-DAPO-17k (Yu et al., 2025) with levels 3-5 of the MATH dataset (Liu et al., 2025), and we evaluate on four standard mathematical-reasoning benchmarks. Experiments are conducted on four model backbones, with GRPO, DAPO, and, where available, GSPO as baselines. The results show that DCPO consistently provides stronger preference-optimization performance, with especially large improvements on the AIME benchmarks: on AIME24-Avg@32 with 32-sample decoding, DCPO-7B scores 38.8 compared with 32.1 (GRPO), 31.6 (DAPO), and 34.9 (GSPO); on AIME25-Avg@32, DCPO-14B reaches 19.0 compared with 10.5 (GRPO), 15.3 (DAPO), and 9.9 (GSPO). DCPO also increases the average nonzero-advantage ratio by 28% over GRPO, doubles training efficiency over DAPO, and reduces token clipping by an order of magnitude relative to the baselines while improving final task performance.

Appendix A.2 provides the preliminary definitions and the loss and advantage formulations for prior algorithms, including GRPO, DAPO, and GSPO.

2 DYNAMIC-ADAPTIVE CLIPPING BOUNDS FOR RL(DAC)

Probability-ratio clipping is a standard stabilization mechanism in reinforcement learning: it limits the magnitude of policy updates by constraining the change in action or token probabilities during optimization. The mechanism is especially relevant in off-policy importance sampling. When samples from the old distribution $q(x)$ are reweighted by the ratio $r(x)$ to estimate expectations under the new distribution $p(x)$, the estimator of $\mathbb{E}_{q(x)}[f(x) \frac{p(x)}{q(x)}]$ remains unbiased, but its variance may increase substantially, as formalized in Equation (1).

$$\text{Var}_{x \sim q} \left[f(x) \frac{p(x)}{q(x)} \right] - \text{Var}_{x \sim p} [f(x)] = \mathbb{E}_{x \sim p} \left[f(x)^2 \left(\frac{p(x)}{q(x)} - 1 \right)^2 \right] = \int f(x)^2 \left(\frac{p(x)}{q(x)} - 1 \right)^2 p(x) dx \quad (1)$$

To control this variance increase, prior work (e.g., Schulman et al. (2017)) constrains the probability ratio within a fixed interval parameterized by the hyperparameter ϵ , as specified in Equation (2).

$$|r(x) - 1| \leq \epsilon \quad (2)$$

A symmetric fixed clipping interval, however, restricts exploration most strongly in regions where the old policy assigns low probability mass. In such regions, the absolute admissible change in probability is small, which limits meaningful updates for rare but potentially informative tokens. DAPO partially relaxes this constraint through asymmetric clipping bounds, but the interval remains fixed rather than probability dependent. GSPO instead removes entire responses when the sequence-level importance ratio violates the clipping interval; this can discard informative tokens while retaining other high-variance tokens, thereby introducing additional instability into training.

From a variance-bias control perspective, the constraint should depend explicitly on the probability mass assigned by the old policy. We therefore impose the probability-dependent restriction in Equation (3).

$$|(r(x) - 1)p(x)| \leq \epsilon \quad (3)$$

Substituting $p(x) = r(x)q(x)$ into Equation (3) gives a closed-form admissible interval for $r(x)$ after enforcing non-negativity of the square-root term through a maximum with zero. The resulting **dynamic-adaptive clipping bounds** are functions of $q(x)$ and the hyperparameters ϵ_{low} and ϵ_{high} , as specified in Equation (4).

$$0.5 + \frac{1}{2} \sqrt{\max\left(1 - \frac{4\epsilon_{low}}{q(x)}, 0\right)} \leq r(x) \leq 0.5 + \frac{1}{2} \sqrt{1 + \frac{4\epsilon_{high}}{q(x)}} \quad (4)$$

This dynamic-adaptive rule expands the admissible update region for low-probability tokens while preserving a constrained update interval for high-probability tokens. Following the motivation of dual clipping (Ye et al., 2020), we cap the maximum ratio for both positive and negative advantages at 10 to prevent the adaptive bounds from producing excessively large updates. Appendix A.5 derives the bounds from the importance-sampling formulation, and Appendix A.6 analyzes their relationship to fixed clipping.

3 SMOOTH ADVANTAGE STANDARDIZATION(SAS)

In GRPO, DAPO, and GSPO, the advantage $\hat{A}_{new,j}^i$ for the j -th response is computed by standardizing only the rewards $R_{j=1,\dots,G}^i$ generated for the same prompt at step i , as in Equation (11) of Appendix A.3. This step-local normalization has two failure modes. First, if sampling randomness yields identical rewards for all responses in a step, the standardized advantage becomes zero, so the prompt contributes no parameter-update signal even when the sampled reasoning trajectories differ. Second, high-entropy sampling can produce highly imbalanced label counts, which induces large step-to-step fluctuations in standardized advantages and can even reverse their signs, thereby destabilizing optimization.

Recent work (Yue et al., 2025) further suggests that RLVR mainly reweights the likelihood of already reachable correct samples rather than constructing an entirely new response distribution. Under this view, responses to the same prompt across training can be treated as observations from a shared global reward distribution, motivating the *cumulative* standardization in Equation (5).

$$\hat{A}_{total,j}^i = \frac{(R_j^i - \mu_{total}^i)}{\sigma_{total}^i} \quad (5)$$

where the normalization statistics are computed over all responses generated for the same prompt up to the current step.

To reduce the variance introduced by both step-specific standardization ($\hat{A}_{new,j}^i$) and cumulative standardization ($\hat{A}_{total,j}^i$), we define two smoothed estimates, $\hat{S}A_{new,j}^i$ and $\hat{S}A_{total,j}^i$. These quantities are weighted averages of the two standardized advantages, with step-dependent weights specified in Equation (6).

$$\hat{S}A_{new,j}^i = \frac{i-1}{i} \hat{A}_{new,j}^i + \frac{1}{i} \hat{A}_{total,j}^i, \quad \hat{S}A_{total,j}^i = \frac{1}{i} \hat{A}_{new,j}^i + \frac{i-1}{i} \hat{A}_{total,j}^i \quad (6)$$

The estimate $\hat{S}A_{new,j}^i$ assigns increasing weight to the current distribution as training proceeds, which prioritizes adaptation to the latest policy outputs. Conversely, $\hat{S}A_{total,j}^i$ assigns increasing weight to the cumulative distribution, interpreting the current-step deviation as a local adjustment within the broader reward history. The final advantage $\hat{A}_j^i$ is selected as the smoothed estimate with smaller absolute magnitude, which limits the effect of fluctuations from either normalization source and improves training stability, as defined in Equation (7).

$$\hat{A}_j^i = \begin{cases} \hat{S}A_{new,j}^i, & \text{when } |\hat{S}A_{new,j}^i| < |\hat{S}A_{total,j}^i| \\ \hat{S}A_{total,j}^i, & \text{otherwise} \end{cases} \quad (7)$$

After a prompt has entered optimization, responses sampled for that prompt in subsequent steps can continue to contribute to parameter updates even when their current step-local advantage is zero. Thus, when a step contains identical rewards, the corresponding responses still update the model through the cumulative component $\frac{1}{i} \hat{A}_{total,j}^i$, preserving useful learning signal and improving data efficiency.

3.1 OTM LOSS

Although the **Token-Level Mean loss (TLM)** (the loss Equation (13) used in DAPO) normalizes token contributions over the batch, it can distort the relative importance of responses with different lengths. Consider two responses to the same prompt: response A has advantage 1 and length 500, whereas response B has advantage 0.5 and length 1500. Under TLM, A receives weight $\frac{500}{500+1500} = 0.25$, and B receives weight $\frac{1500}{500+1500} = 0.75$. The resulting aggregate contribution is therefore $1 \times 0.25 = 0.25$ for A and $0.5 \times 0.75 = 0.375$ for B , so the lower-advantage response exerts the larger loss contribution.

Conversely, the **Sequence-Level Mean loss (SLM)** (the loss Equation (12) used in GRPO) averages across the response group and thereby scales the effective advantage magnitude by approximately G^{-1} after advantage computation. For large batches, this operation can attenuate the standardized response-level relationships encoded by the advantages. Because standardized advantages already define relative preferences among responses, averaging only over the tokens of each response using $|o_i|$ preserves those response-level relations while distributing the response contribution uniformly across its tokens. We therefore average tokens within each response but do not average across responses in the same batch; this objective is the **Only Token Mean loss (OTM)** in Equation (8).

$$\mathcal{T}_{\text{DCPO}}(\theta) = \sum_{i=1}^G \frac{1}{|o_i|} \sum_{t=1}^{|o_i|} \min \left(r_{i,t}(\theta) \hat{A}_{i,t}, \text{clip}(r_{i,t}(\theta), 1 - \varepsilon_{\text{low}}, 1 + \varepsilon_{\text{high}}) \hat{A}_{i,t} \right) \quad (8)$$

As in DAPO, the DCPO objective does not include an explicit KL-divergence constraint. The lower and upper clipping limits in the objective are instantiated with the dynamic clipping bounds introduced in Section 2.

4 EXPERIMENTAL SETUP

Models and Datasets We evaluate DCPO on four model backbones: Qwen2.5-Math-1.5B-Instruct, Qwen2.5-3B (common base), Qwen2.5-Math-7B (math base), and Qwen2.5-14B (common base) (Yang et al., 2024a;b). The training corpus contains approximately 25k mathematical problems obtained by merging DAPO-Math-17K (Yu et al., 2025) with levels 3-5 of the MATH subdataset (Hendrycks et al., 2021b; Liu et al., 2025). For both training and evaluation, each problem is formatted with the Qwen-Math template described in Section A.8.

Evaluation Evaluation is conducted on four mathematical-reasoning benchmarks: MATH500 (Hendrycks et al., 2021a), AMC23 (Ouyang et al., 2022), AIME24 (Li et al., 2024), and AIME25 (MAA). For **MATH500**, which contains 500 problems, we report only Avg@1. For the smaller benchmarks, **AMC23** (40 problems), **AIME24** (30 problems), and **AIME25** (30 problems), we report both Avg@1 and Avg@32 to characterize deterministic accuracy and sampling robustness.

- **Avg@1:** This metric is the accuracy obtained under greedy decoding and measures the model’s deterministic best-response performance.
- **Avg@32:** This metric is the mean accuracy over 32 sampled responses per problem, generated with temperature 1.0 and top_p 1.0, and measures the robustness and stability of the learned policy distribution under stochastic decoding.

Baseline Settings We use GRPO and DAPO as primary baselines and include GSPO as an additional baseline for Qwen2.5-Math-7B and Qwen2.5-14B. Implementation details, clipping thresholds ϵ_* , codebase information, and shared training hyperparameters are specified in Appendix A.9.

5 RESULTS AND ANALYSIS

5.1 MAIN RESULTS

Table 1: Benchmark performance under greedy and stochastic decoding. Avg@1 denotes greedy-decoding accuracy; Avg@32 denotes mean accuracy over 32 sampled responses. Bold entries indicate the best value in each comparison block, and Δ reports the difference relative to each baseline.

Model	MATH500 (Avg@1)	AMC23 (Avg@1/32)	AIME24 (Avg@1/32)	AIME25 (Avg@1/32)	Average (Avg@1/32)
Qwen2.5-Math-1.5B-Instruct					
base	73.6	57.5/49.4	10.0/10.0	3.3/6.1	36.1/21.8
GRPO	77.2	70.0/68.4	16.7/14.0	20.0/13.5	46.0/32.0
DAPO	76.0	80.0/70.6	20.0/13.5	14.4/12.5	46.5/32.4
DCPO	77.2	75.0/ 70.8	20.0/15.6	16.7/12.1	47.2/32.8
Δ_{GRPO}	+0.0	+5.0/+2.4	+3.3/+1.6	-3.3/-1.4	+1.2/+0.8
Δ_{DAPO}	+1.2	-5.0/ +0.2	+0.0/+2.1	+2.3/-0.4	+0.7/+0.4
Qwen2.5-3B[Base]					
base	46.4	27.5/7.8	3.3/0.1	3.3/0.7	20.1/2.9
GRPO	69.2	62.5/51.6	10.0/7.5	6.7/4.5	36.3/21.0
DAPO	72.4	57.5/54.0	10.0/8.3	6.7/3.9	36.6/23.1
DCPO	71.2	62.5/55.8	3.3/7.5	10.0/4.7	37.6/22.7
Δ_{GRPO}	+2.0	+0.0/+4.2	-6.7/ +0.0	+3.3/+0.2	+1.3/+1.7
Δ_{DAPO}	-1.2	+5.0/+1.8	-6.7/-0.8	+3.3/+0.8	+1.0/-0.4
Qwen2.5-Math-7B[Base]					
base	50.4	40.0/19.5	13.3/6.0	3.3/1.5	28.4/9.3
GRPO	81.6	77.5/75.9	36.7/32.1	16.7/16.7	53.1/41.6
DAPO	83.0	72.5/ 80.7	36.7/31.6	23.3/14.9	53.9/42.4
GSPO	84.0	80.0/78.8	40.0/34.9	16.7/16.2	55.2/43.3
DCPO	82.5	82.6/79.8	46.7/38.8	16.7/ 17.2	57.1/45.2
Δ_{GRPO}	+0.9	+5.1/+4.9	+10.0/+6.7	+0.0/+0.5	+4.0/+3.6
Δ_{DAPO}	-0.5	+10.1/-0.9	+10.0/+7.2	-6.6/ +2.3	+3.3/+2.8
Δ_{GSPO}	-1.5	+2.6/+1.0	+6.7/+3.9	+0.0/+1.0	+1.9/+1.9
Qwen2.5-14B[Base]					
base	60.8	47.5/16.4	3.3/1.3	3.3/1.1	28.7/6.3
GRPO	81.2	75.0/65.6	13.3/17.6	13.3/10.5	45.7/31.3
DAPO	83.4	87.5/85.1	16.7/16.4	20.0/15.3	51.9/38.9
GSPO	78.6	77.5/75.0	23.3/16.0	16.7/9.9	49.0/33.5
DCPO	84.6	85.0/79.9	20.0/ 18.2	23.3/19.0	53.2/39.0
Δ_{GRPO}	+3.4	+10.0/+14.3	+6.7/+0.6	+10.0/+8.5	+6.5/+7.7
Δ_{DAPO}	+1.2	-2.5/-5.2	+3.3/+1.8	+3.3/+3.7	+1.3/+0.1
Δ_{GSPO}	+6.0	+7.5/4.9	-3.3/ +2.2	+6.6/+10.1	+4.2/+5.5

Table 1 reports the performance of GRPO, DAPO, GSPO, and DCPO on MATH500, AMC23, AIME24, and AIME25. The table includes greedy-decoding accuracy (Avg@1) and sampling-based accuracy (Avg@32), so the comparison measures both deterministic problem solving and robustness under stochastic generation. Across model scales, DCPO achieves consistently superior or comparable results relative to GRPO, DAPO, and, where evaluated, GSPO. This pattern indicates that the proposed objective functions as a general preference-optimization procedure for RLVR training of LLMs.

Overall Superiority of DCPO For Qwen2.5-Math-1.5B-Instruct, DCPO obtains the highest average scores, (47.2/32.8), improving over GRPO by (+1.2/+0.8) and over DAPO by (+0.7/+0.4). For Qwen2.5-3B, DCPO reaches (37.6/22.7), exceeding GRPO by (+1.3/+1.7) and remaining competitive with DAPO, with an Avg@1 gain of (+1.0) and an Avg@32 difference of only (-0.4). For Qwen2.5-Math-7B, DCPO attains (57.1/45.2), improving over GRPO by (+4.0/+3.6), DAPO by (+3.3/+2.8), and GSPO by (+1.9/+1.9). For Qwen2.5-14B, the largest evaluated model, DCPO achieves (53.2/39.0), yielding substantial gains over GRPO (+6.5/+7.7) and improvements over

DAPO (+1.3/+0.1) and GSPO (+4.2/+5.5). These results show that DCPO maintains effective reward alignment across model capacities and across both greedy and stochastic decoding regimes.

Robustness on AIME under Sampling (Avg@32) A particularly informative pattern appears on AIME24 and AIME25, which are high-difficulty competition-style mathematics benchmarks that stress multi-step reasoning. Under Avg@32, which measures whether the trained policy can produce correct responses under high-entropy sampling, DCPO is substantially more robust at the 7B and 14B scales. On AIME24-Avg@32, DCPO-7B scores 38.8, exceeding GRPO (32.1), DAPO (31.6), and GSPO (34.9). On AIME25-Avg@32, DCPO-14B reaches 19.0, exceeding GRPO (10.5), DAPO (15.3), and GSPO (9.9). These gains indicate that DCPO improves not only average accuracy but also the reliability of correct generations under sampling-based decoding, where temperature-induced variability requires a policy to preserve multiple plausible reasoning trajectories.

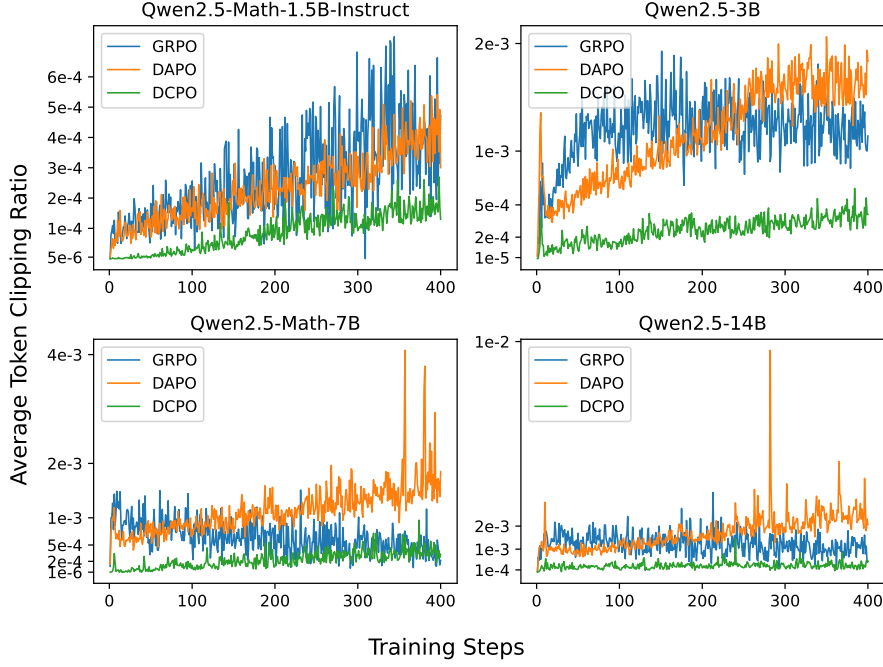


Figure 1: *TCR* trajectories across model backbones and optimization methods.

5.2 THE TOKEN CLIPPING RATIO (*TCR*)

We define the **Token Clipping Ratio** (*TCR*) as the fraction of generated tokens excluded from policy-gradient updates during back-propagation because their probability ratios are clipped. Figure 1 shows that *TCR* exhibits method-specific dynamics across training.

$$\text{TCR} = \text{Average} \left(\sum_{m=1}^N \frac{\text{Number of clipped tokens in } \textit{micro}_m}{\text{Total number of tokens in } \textit{micro}_m} \right) \quad (9)$$

where $\textit{micro}_m$ denotes the m -th microbatch and N denotes the number of microbatches in one training step.

Under GRPO, *TCR* depends strongly on model scale: it increases over training for the smaller Qwen2.5-Math-1.5B and Qwen2.5-3B models, whereas it gradually decreases for Qwen2.5-Math-7B and Qwen2.5-14B. DAPO instead exhibits a consistent upward *TCR* trajectory across all model sizes. This behavior implies that an increasing fraction of tokens is removed from DAPO updates as training progresses, so more updates are computed from partially clipped response sequences. Both GRPO and DAPO also show substantial fluctuations and occasional spikes in *TCR*, which can introduce optimization instability and contribute to entropy collapse. For GSPO, the average *TCR* exceeds 11% on Qwen2.5-Math-7B and 15% on Qwen2.5-14B; these values are substantially higher than those of token-level clipping methods and correspond to greater loss of informative tokens.

DCPO, by contrast, maintains a stable and low-clipping regime. Across model sizes and training stages, its TCR remains approximately constant and is an order of magnitude lower than the corresponding GRPO and DAPO values. A lower and more stable TCR means that DCPO discards fewer tokens, permits a larger portion of each generated response to contribute to the update, and allocates more effective exploration capacity to rare-token regions while still constraining extreme ratio values.

For Qwen2.5-Math-7B, the token distribution rapidly concentrates on high-confidence tokens: after approximately 60 steps, about 95% of generated tokens satisfy ($q(x) > 0.9$); after approximately 100 steps, this fraction rises to about 97% and continues increasing later in training. As illustrated in Figure 4d, once the old probability satisfies ($q(x) \geq \frac{1}{1+\epsilon_{grpo}} \approx 0.83$), the new probability $p(x)$ remains within the update-admissible interval $[\frac{1}{1+\epsilon}, 1]$ and is not discarded by the clipping rule. Consequently, most tokens fall in a non-clipped region, which explains the low aggregate TCR observed empirically. At the same time, Figure 1 shows that TCR increases in the later phases of many runs, indicating that a growing fraction of low-probability tokens can be excluded and that rare-token exploration may become increasingly restricted.

Previous work (Wang et al., 2025) shows that **high-entropy tokens**, which correspond to low-probability events, are central to the emergence of reasoning capabilities. Because lower-probability tokens generally carry higher information content, expanding the admissible update range for such tokens increases the model’s effective exploration space and supports the acquisition of more diverse reasoning trajectories.

These empirical and theoretical observations motivate the proposed **dynamic-adaptive clipping** scheme, which relaxes clipping for tokens satisfying $q(x) < \frac{1}{1+\epsilon}$ while preserving the conservative interval for high-confidence tokens. By expanding the update interval for the high-entropy tail of the distribution, the method enables learning from rare informative tokens and thereby improves reasoning performance.

5.3 THE RESPONSE UTILIZATION RATIO (RUR)

During RLVR training, a generated response contributes to parameter updates only when its advantage induces a nonzero gradient. We quantify this response-level efficiency with the **Response Utilization Ratio** (RUR), defined as the percentage of generated responses with nonzero advantage that participate in the policy update.

$$RUR = \frac{\text{Number of responses with non-zero advantage}}{\text{Total number of generated responses}} \times 100\% \quad (10)$$

Table 2 reports that the average RUR during GRPO training is low, with an overall mean of 44.6% across four models. Thus, more than half of generated responses fail to provide update signal, indicating substantial inefficiency in response-level exploration. DCPO achieves an average RUR of approximately 70% after the first epoch, corresponding to an absolute increase of 28% and a relative improvement of 64% over GRPO.

Table 2: Average RUR measured over 400 training steps.

model	GRPO	GSPO	DCPO
Qwen2.5-Math-1.5B-Instruct	45.6%	-	67.1%
Qwen2.5-3B	48.3%	-	74.3%
Qwen2.5-Math-7B	37.4%	43.5%	73.2%
Qwen2.5-14B	43.9%	47.6%	72.4%
Average	43.8%	45.6%	71.8%

Figure 2 shows that GRPO’s RUR decreases substantially over time for all evaluated model types and sizes. In particular, RUR falls from above 90% to below 50%; for Qwen2.5-Math-7B, it reaches approximately 30% after about 200 training steps. This trend indicates that GRPO’s current-step advantage standardization produces a large fraction of zero-gradient responses, making late-stage response exploration inefficient. In contrast, DCPO maintains a consistently higher RUR through smooth advantage standardization. After the first epoch, RUR stabilizes near 70% and continues to improve as training proceeds.

The observed RUR exceeds 90% during the first few steps, but from the second epoch through the end of training it remains near 70% rather than reaching 100%. This behavior is consistent with the intended filtering effect of the SAS mechanism.

First, the initial *RUR* above 90% is primarily caused by early model outputs that violate the required format, receive negative rewards, and thereby inflate the count of nonzero-advantage responses. This suggests that models rapidly acquire the fixed answer format, after which training should emphasize reasoning-capability exploration. Second, two mechanisms explain why some advantages remain zero and why *RUR* stabilizes near 70% instead of 100%: (i) Nearly half of the math problems are sufficiently easy that the model solves them consistently, so excluding them from further updates is reasonable because the corresponding capability does not require additional refinement. (ii) The remaining zero-advantage instances are consistently erroneous. This pattern can arise for two reasons: some problems are too difficult for the current model even when the ground-truth label is correct. In addition, DAPO-Math-17K contains some examples whose labels were converted into mislabeled integers, introducing potential label noise. Discarding zero-advantage responses in these cases functions as a filtering mechanism and reduces the risk of updating the model on noisy or incorrectly labeled data.

Unlike GRPO and DCPO, DAPO removes generated responses with identical labels, which makes direct *RUR* comparison ill-posed. We therefore evaluate DAPO efficiency by measuring the number of generated responses and the wall-clock training time required to complete the same number of update steps. To compensate for the low sampling efficiency associated with low *RUR*, we set the DAPO generation batch size to 1–4 times the training batch size and adjust it dynamically. Across four models, DAPO requires 3 to 5 times more generated responses than DCPO to perform the same number of parameter updates, which at least doubles training GPU hours without improving final performance. DAPO’s data utilization also decreases with model size; for Qwen2.5-14B, the fraction of responses retained by DAPO falls below 30% after roughly 300 steps. These results indicate that DAPO’s dynamic sampling strategy has limited data utilization and training efficiency and can forgo useful exploration of response diversity.

In summary, DCPO improves over GRPO and DAPO by increasing response utilization, reducing generation waste, and preserving more sampled trajectories for diversity exploration, which jointly improves training efficiency and downstream performance.

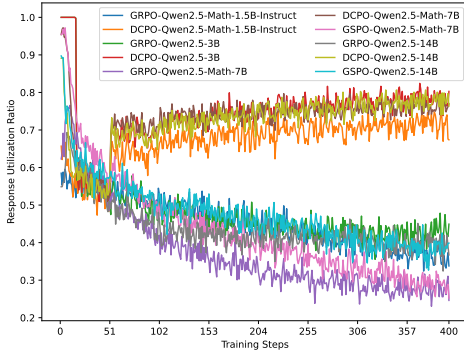


Figure 2: Evolution of *RUR* during training.

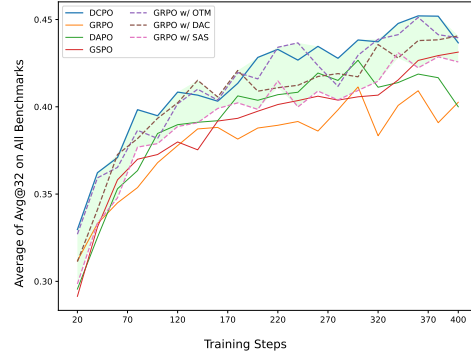


Figure 3: Ablation results using average Avg@32 on Qwen2.5-Math-7B.

5.4 ABLATION STUDIES

We conduct component-level ablations on Qwen2.5-Math-7B to quantify the contribution of each DCPO mechanism, using **Avg@32** as the evaluation metric because it reflects the robustness and stability of the learned policy distribution. For a controlled comparison, each ablation modifies one component of the GRPO baseline while keeping all other settings fixed; the KL-divergence term is removed to align the setup with DAPO, GSPO, and full DCPO. Figure 3 shows the learning curves from 20 to 400 steps.

The learning curves show that each individual component improves inference performance over training. The ablated variants surpass GRPO and reach or exceed the performance of DAPO and GSPO, indicating that each mechanism contributes an independent optimization benefit. In later training stages, baseline performance plateaus, whereas DCPO and its component variants continue to improve. These patterns indicate that the three mechanisms provide complementary gains in data efficiency, stability, and exploration. The component-level effects are summarized as follows.

(i) **Only-Token-Mean loss (OTM)**: Replacing batch-level aggregation with within-response token averaging improves the GRPO baseline and reduces the adverse effects of low data utilization and unstable response weighting. (ii) **Smoothed Advantage Standardization (SAS)**: Replacing stepwise advantage normalization with cumulative smoothed advantages substantially improves GRPO and approaches DAPO performance, showing that reward statistics aggregated across training steps stabilize optimization and improve final accuracy. (iii) **Dynamic Adaptive Clipping (DAC)**: Replacing GRPO’s fixed clipping interval with a probability-adaptive interval yields large gains. DAC alone surpasses GRPO, DAPO, and GSPO and is more stable than the OTM-only variant. (iv) **Combination of OTM + SAS + DAC (full DCPO)**: Combining the three mechanisms produces the strongest and most stable results, substantially outperforming GRPO, DAPO, and GSPO. The combined effect indicates that OTM, SAS, and DAC address distinct sources of inefficiency in RLVR optimization.

Overall, the ablation results verify that each DCPO component contributes positively and that their combination yields cumulative improvements. The evidence supports the proposed mechanisms as effective tools for improving data efficiency and stability in reinforcement learning for LLMs.

6 CONCLUSION

This work addresses two limitations of existing RLVR policy-optimization methods: restricted token-level exploration caused by fixed clipping in low-probability regions, and low sample efficiency caused by current-step reward standardization. We introduce **Dynamic Clipping Policy Optimization (DCPO)**, a reinforcement-learning pipeline for improving LLM reasoning through two principal mechanisms: (i) **dynamically adaptive token-level clipping**, which sets clipping bounds as functions of old-policy token probabilities and thereby improves exploration of rare low-probability tokens; and (ii) **Cumulative-Smooth Advantage Standardization**, which aggregates reward statistics across optimization steps and reduces the variance induced by high-entropy sampling. Experiments on four mathematical-reasoning benchmarks and four model backbones demonstrate the effectiveness of DCPO. For example, DCPO-7B achieves an Avg@32 score of 38.8 on AIME24, outperforming GRPO (32.1) and DAPO (31.6). DCPO also increases the response utilization ratio by 28% relative to GRPO, doubles training efficiency relative to DAPO, and reduces the token clipping ratio by an order of magnitude. These findings show that DCPO improves both data utilization and diversity-preserving exploration in RLVR. Future work will evaluate whether the same mechanisms transfer to other domains, including code generation and semantic reasoning.

REFERENCES

- Gheorghe Comanici, Eric Bieber, Mike Schaekermann, Ice Pasupat, Naveen Sachdeva, Inderjit Dhillon, Marcel Blistein, Ori Ram, Dan Zhang, Evan Rosen, et al. Gemini 2.5: Pushing the frontier with advanced reasoning, multimodality, long context, and next generation agentic capabilities. *arXiv preprint arXiv:2507.06261*, 2025.
- DeepSeek-AI, Aixin Liu, Bei Feng, Bing Xue, Bingxuan Wang, Bochao Wu, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan, Damai Dai, Daya Guo, Dejian Yang, Deli Chen, Dongjie Ji, Erhang Li, Fangyun Lin, Fucong Dai, Fuli Luo, Guangbo Hao, Guanting Chen, Guowei Li, H. Zhang, Han Bao, Hanwei Xu, Haocheng Wang, Haowei Zhang, Honghui Ding, Huajian Xin, Huazuo Gao, Hui Li, Hui Qu, J. L. Cai, Jian Liang, Jianzhong Guo, Jiaqi Ni, Jiashi Li, Jiawei Wang, Jin Chen, Jingchang Chen, Jingyang Yuan, Junjie Qiu, Junlong Li, Junxiao Song, Kai Dong, Kai Hu, Kaige Gao, Kang Guan, Kexin Huang, Kuai Yu, Lean Wang, Lecong Zhang, Lei Xu, Leyi Xia, Liang Zhao, Litong Wang, Liyue Zhang, Meng Li, Miaojuan Wang, Mingchuan Zhang, Minghua Zhang, Minghui Tang, Mingming Li, Ning Tian, Panpan Huang, Peiyi Wang, Peng Zhang, Qiancheng Wang, Qihao Zhu, Qinyu Chen, Qiushi Du, R. J. Chen, R. L. Jin, Ruiqi Ge, Ruisong Zhang, Ruizhe Pan, Runji Wang, Runxin Xu, Ruoyu Zhang, Ruyi Chen, S. S. Li, Shanghao Lu, Shangyan Zhou, Shanhuang Chen, Shaoqing Wu, Shengfeng Ye, Shengfeng Ye, Shirong Ma, Shiyu Wang, Shuang Zhou, Shuiping Yu, Shunfeng Zhou, Shuting Pan, T. Wang, Tao Yun, Tian Pei, Tianyu Sun, W. L. Xiao, Wangding Zeng, Wanjia Zhao, Wei An, Wen Liu, Wenfeng Liang, Wenjun Gao, Wenqin Yu, Wentao Zhang, X. Q. Li, Xiangyue Jin, Xianzu Wang, Xiao Bi, Xiaodong Liu, Xiaohan Wang, Xiaojin Shen, Xiaokang Chen, Xiaokang Zhang, Xiaosha Chen, Xiaotao Nie, Xiaowen Sun, Xiaoxiang Wang, Xin Cheng, Xin Liu, Xin Xie, Xingchao Liu,

- Xingkai Yu, Xinnan Song, Xinxia Shan, Xinyi Zhou, Xinyu Yang, Xinyuan Li, Xuecheng Su, Xuheng Lin, Y. K. Li, Y. Q. Wang, Y. X. Wei, Y. X. Zhu, Yang Zhang, Yanhong Xu, Yanhong Xu, Yanping Huang, Yao Li, Yao Zhao, Yaofeng Sun, Yaohui Li, Yaohui Wang, Yi Yu, Yi Zheng, Yichao Zhang, Yifan Shi, Yiliang Xiong, Ying He, Ying Tang, Yishi Piao, Yisong Wang, Yixuan Tan, Yiyang Ma, Yiyuan Liu, Yongqiang Guo, Yu Wu, Yuan Ou, Yuchen Zhu, Yuduan Wang, Yue Gong, Yuheng Zou, Yujia He, Yukun Zha, Yunfan Xiong, Yunxian Ma, Yuting Yan, Yuxiang Luo, Yuxiang You, Yuxuan Liu, Yuyang Zhou, Z. F. Wu, Z. Z. Ren, Zehui Ren, Zhangli Sha, Zhe Fu, Zhean Xu, Zhen Huang, Zhen Zhang, Zhenda Xie, Zhengyan Zhang, Zhewen Hao, Zhibin Gou, Zhicheng Ma, Zhigang Yan, Zhihong Shao, Zhipeng Xu, Zhiyu Wu, Zhongyu Zhang, Zhuoshu Li, Zihui Gu, Zijia Zhu, Zijun Liu, Zilin Li, Ziwei Xie, Ziyang Song, Ziyi Gao, and Zizheng Pan. Deepseek-v3 technical report, 2024. URL <https://arxiv.org/abs/2412.19437>.
- Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, Runxin Xu, Qihao Zhu, Shirong Ma, Peiyi Wang, Xiao Bi, et al. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025.
- Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. Measuring mathematical problem solving with the math dataset. *arXiv preprint arXiv:2103.03874*, 2021a.
- Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. Measuring mathematical problem solving with the math dataset. *Sort*, 2(4): 0–6, 2021b.
- Jia Li, Edward Beeching, Lewis Tunstall, Ben Lipkin, Roman Soletskyi, Shengyi Huang, Kashif Rasul, Longhui Yu, Albert Q Jiang, Ziju Shen, et al. Numinamath: The largest public dataset in ai4maths with 860k pairs of competition math problems and solutions. *Hugging Face repository*, 13:9, 2024.
- Zichen Liu, Changyu Chen, Wenjun Li, Penghui Qi, Tianyu Pang, Chao Du, Wee Sun Lee, and Min Lin. Understanding r1-zero-like training: A critical perspective. *arXiv preprint arXiv:2503.20783*, 2025.
- MAA. American invitational mathematics examination (AIME). Mathematics Competition Series, 2024. URL <https://maa.org/math-competitions/aime>.
- OpenAI. Introducing openai o3 and o4-mini. 2025. URL <https://openai.com/index/introducing-o3-and-o4-mini/>.
- Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. Training language models to follow instructions with human feedback. *Advances in neural information processing systems*, 35:27730–27744, 2022.
- John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.
- Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, YK Li, Yang Wu, et al. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.
- Guangming Sheng, Chi Zhang, Zilingfeng Ye, Xibin Wu, Wang Zhang, Ru Zhang, Yanghua Peng, Haibin Lin, and Chuan Wu. Hybridflow: A flexible and efficient rlhf framework. *arXiv preprint arXiv:2409.19256*, 2024.
- Shenzhi Wang, Le Yu, Chang Gao, Chujie Zheng, Shixuan Liu, Rui Lu, Kai Dang, Xionghui Chen, Jianxin Yang, Zhenru Zhang, et al. Beyond the 80/20 rule: High-entropy minority tokens drive effective reinforcement learning for llm reasoning. *arXiv preprint arXiv:2506.01939*, 2025.
- An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chengyuan Li, Dayiheng Liu, Fei Huang, Haoran Wei, et al. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*, 2024a.

- An Yang, Beichen Zhang, Binyuan Hui, Bofei Gao, Bowen Yu, Chengpeng Li, Dayiheng Liu, Jianhong Tu, Jingren Zhou, Junyang Lin, et al. Qwen2.5-math technical report: Toward mathematical expert model via self-improvement. *arXiv preprint arXiv:2409.12122*, 2024b.
- Deheng Ye, Zhao Liu, Mingfei Sun, Bei Shi, Peilin Zhao, Hao Wu, Hongsheng Yu, Shaojie Yang, Xipeng Wu, Qingwei Guo, et al. Mastering complex control in moba games with deep reinforcement learning. In *Proceedings of the AAAI conference on artificial intelligence*, volume 34, pp. 6672–6679, 2020.
- Qiyang Yu, Zheng Zhang, Ruofei Zhu, Yufeng Yuan, Xiaochen Zuo, Yu Yue, Tiantian Fan, Gaohong Liu, Lingjun Liu, Xin Liu, et al. Dapo: An open-source llm reinforcement learning system at scale. *arXiv preprint arXiv:2503.14476*, 2025.
- Yufeng Yuan, Qiyang Yu, Xiaochen Zuo, Ruofei Zhu, Wenyuan Xu, Jiaze Chen, Chengyi Wang, Tiantian Fan, Zhengyin Du, Xiangpeng Wei, et al. Vapo: Efficient and reliable reinforcement learning for advanced reasoning tasks. *arXiv preprint arXiv:2504.05118*, 2025.
- Yang Yue, Zhiqi Chen, Rui Lu, Andrew Zhao, Zhaokai Wang, Shiji Song, and Gao Huang. Does reinforcement learning really incentivize reasoning capacity in llms beyond the base model? *arXiv preprint arXiv:2504.13837*, 2025.
- Chujie Zheng, Shixuan Liu, Mingze Li, Xiong-Hui Chen, Bowen Yu, Chang Gao, Kai Dang, Yuqiong Liu, Rui Men, An Yang, et al. Group sequence policy optimization. *arXiv preprint arXiv:2507.18071*, 2025.

A APPENDIX

A.1 MAIN SYMBOL DEFINITIONS

G	Number of responses generated per prompt at each step.
θ	Parameters of the actor model.
$\pi_{\theta}(o_i q)$	Probability of response o_i given prompt q under parameters θ .
$\pi_{\theta_{old}}(o_i q)$	Probability of response o_i given prompt q under previous parameters θ_{old} .
π_{ref}	Reference policy for KL divergence computation.
β	Coefficient for KL divergence term (controls policy update vs. constraint trade-off).
$r_{i,t}(\theta)$	Probability ratio $\frac{\pi_{\theta}(o_i^t q, o_i^{1:t-1})}{\pi_{\theta_{old}}(o_i^t q, o_i^{1:t-1})}$ for token t in response o_i .
$\hat{A}_{i,t}$	Standardized advantage estimate for token t in response o_i .
$ o_i $	The length of the response in tokens.
$\mathbb{D}_{KL}(\pi_{\theta} \pi_{ref})$	KL divergence between current and reference policies.
ε	Probability ratio clipping threshold.
ε_{low} and ε_{high}	Lower/upper thresholds for probability ratio clipping.
M	Maximum allowed response length.
$is_equivalent(a, o_i)$	Binary function indicating whether response o_i matches answer a .
μ_{new}^i	Mean reward across G responses generated at step i (0 if $i = 0$).
σ_{new}^i	Reward variance across G responses at step i (0 if $i = 0$).
μ_{old}^i	Mean reward across $G \cdot (i - 1)$ responses from steps 1 to $i - 1$ (0 if $i \leq 1$).
σ_{old}^i	Reward variance across $G \cdot (i - 1)$ responses from steps 1 to $i - 1$ (0 if $i \leq 1$).
μ_{total}^i	Mean reward across $G \cdot i$ responses from steps 1 to i (0 if $i = 0$).
σ_{total}^i	Reward variance across $G \cdot i$ responses from steps 1 to i (0 if $i = 0$).

A.2 PRELIMINARY

A.3 ADVANTAGE CALCULATION

In GRPO and DAPO, the token-level advantage $\hat{A}_{j,t}^i$ for token t in response j is obtained by standardizing the response reward R_j^i with respect to the mean μ^i and standard deviation σ^i of the G responses generated at training step i , as shown in Equation (11).

$$\hat{A}_{j,t}^i = \frac{(R_j^i - \mu^i)}{\sigma^i} \quad (11)$$

When all responses to the same prompt receive identical rewards, the standardized advantages become zero and the corresponding responses do not contribute to the model update, which wastes generated data.

A.3.1 GRPO

Shao et al. (2024) proposed **Group Relative Policy Optimization (GRPO)**. GRPO samples G responses for each query, assigns rewards R with a rule-based reward function, and estimates token-level advantages according to Equation (11). The actor parameters are then updated with the **Sequence Level Mean loss (SLM)** in Equation (12):

$$\mathcal{T}_{\text{GRPO}}(\theta) = \frac{1}{G} \sum_{i=1}^G \frac{1}{|o_i|} \sum_{t=1}^{|o_i|} \min \left(r_{i,t}(\theta) \hat{A}_{i,t}, \text{clip}(r_{i,t}(\theta), 1 - \varepsilon, 1 + \varepsilon) \hat{A}_{i,t} \right) - \beta \mathbb{D}_{\text{KL}}(\pi_\theta \| \pi_{\text{ref}}) \quad (12)$$

The first term averages token-level contributions within each response and then averages across the G responses in the batch. The $\mathbb{D}_{\text{KL}}$ penalty constrains the learned policy relative to a reference policy, such as a pre-trained model. The fixed clipping threshold ε limits large updates by bounding token probability ratios. Although this averaging strategy is widely used, it attenuates the contribution of individual responses.

A.3.2 DAPO

Yu et al. (2025) proposed **Dynamic sAmpling Policy Optimization (DAPO)** after observing that longer GRPO sequences can exert disproportionate influence on gradient updates. DAPO normalizes by the total token count over the batch, yielding the **Token Level Mean loss (TLM)** in Equation (13).

$$\mathcal{T}_{\text{DAPO}}(\theta) = \frac{1}{\sum_{i=1}^G |o_i|} \sum_{i=1}^G \sum_{t=1}^{|o_i|} \min \left(r_{i,t}(\theta) \hat{A}_{i,t}, \text{clip}(r_{i,t}(\theta), 1 - \varepsilon_{\text{low}}, 1 + \varepsilon_{\text{high}}) \hat{A}_{i,t} \right) \quad (13)$$

subject to $0 < |\{o_i \mid \text{is_equivalent}(a, o_i)\}| < G$. Under this constraint, prompts whose responses all receive identical rewards are discarded and regenerated to maintain batch size, which introduces substantial data inefficiency.

A.3.3 GSPO

Zheng et al. (2025) proposed **Group Sequence Policy Optimization (GSPO)**, arguing that token-level clipping can introduce high-variance noise into gradient estimates. This noise can accumulate over long sequences and can be amplified by the clipping operation. GSPO mitigates this issue by replacing token-wise clipping with sequence-wise clipping, thereby producing a more stable training signal.

$$\mathcal{T}_{\text{GSPO}}(\theta) = \frac{1}{G} \sum_{i=1}^G \min \left(s_i(\theta) \hat{A}_i, \text{clip}(s_i(\theta), 1 - \varepsilon, 1 + \varepsilon) \hat{A}_i \right) \quad (14)$$

$$\text{where } s_i(\theta) = \exp \left(\frac{1}{|o_i|} \sum_{t=1}^{|o_i|} \log \left(\frac{\pi_\theta(o_{j,t}|q)}{\pi_{\theta_{\text{old}}}(o_{j,t}|q)} \right) \right)$$

GSPO discards more than 10% of responses with nonzero advantages, which increases data waste and reduces training efficiency. Although sequence-level clipping removes responses with high sequence-level variance, many retained responses can still contain high-entropy tokens, which may preserve token-level instability.

A.4 REWARD CALCULATION

We use a rule-based reward model to provide a verifiable and interpretable supervision signal. Each generated response is evaluated according to answer correctness and format compliance, as specified in Equation (15): a reward of +1 is assigned when both the format and answer are correct, a reward of 0 is assigned when the format is correct but the answer is incorrect, and a reward of -1 is assigned when the format is incorrect.

$$R_j^i = \begin{cases} 1, & \text{correct format and correct answer} \\ 0, & \text{correct format but incorrect answer} \\ -1, & \text{incorrect format} \end{cases} \quad (15)$$

where j indexes the response group for a given problem at training step i , and the reward is determined solely by the response format and answer correctness.

A.5 HYPOTHETICAL REASONING PROCESS FOR DYNAMIC-ADAPTIVE CLIPPING BOUNDS

Probability-ratio clipping is a central stabilization technique in reinforcement learning because it restricts large policy updates that can destabilize optimization. The mechanism is especially important when importance sampling is used to estimate expectations under a new policy from samples generated by an old policy.

The expectation of a function $f(x)$ under the new distribution $p(x)$ can be rewritten as an expectation under the old distribution $q(x)$ by applying the importance weight $\frac{p(x)}{q(x)}$, as shown in equation 16:

$$\int f(x)p(x) dx = \int f(x)\frac{p(x)}{q(x)}q(x) dx = \mathbb{E}_{x \sim q} \left[f(x)\frac{p(x)}{q(x)} \right] \quad (16)$$

Although the resulting estimator is unbiased, its variance can be substantially inflated, which is the principal risk in importance sampling. The true variance of $f(x)$ under the new policy is given in Equation (17), whereas the variance of the importance-sampled estimator under the old policy is given in Equation (18):

$$\text{Var}_{x \sim p}[f(x)] = \mathbb{E}_{x \sim p}[f(x)^2] - (\mathbb{E}_{x \sim p}[f(x)])^2 \quad (17)$$

$$\begin{aligned} \text{Var}_{x \sim q} \left[f(x)\frac{p(x)}{q(x)} \right] &= \mathbb{E}_{x \sim q} \left[\left(f(x)\frac{p(x)}{q(x)} \right)^2 \right] - \left(\mathbb{E}_{x \sim q} \left[f(x)\frac{p(x)}{q(x)} \right] \right)^2 \\ &= \mathbb{E}_{x \sim p} \left[f(x)^2 \frac{p(x)}{q(x)} \right] - (\mathbb{E}_{x \sim p}[f(x)])^2 \end{aligned} \quad (18)$$

The difference between the importance-sampled variance and the true variance quantifies the inflation term, as shown in Equation (19):

$$\text{Var}_{x \sim q} \left[f(x)\frac{p(x)}{q(x)} \right] - \text{Var}_{x \sim p}[f(x)] = \mathbb{E}_{x \sim p} \left[f(x)^2 \left(\frac{p(x)}{q(x)} - 1 \right) \right] = \int f(x)^2 \left(\frac{p(x)}{q(x)} - 1 \right) p(x) dx \quad (19)$$

To reduce this inflation, many prior methods, including PPO Schulman et al. (2017) and RLVR algorithms (DeepSeek-AI et al., 2024; Liu et al., 2025), apply a fixed symmetric clipping bound to the probability ratio $r(x) = \frac{p(x)}{q(x)}$, as defined in Equation (20):

$$|r(x) - 1| \leq \epsilon \quad (20)$$

However, a fixed clipping interval inherently limits the policy’s exploration capacity, especially in regions where the old policy assigns low probability mass. In these regions, the admissible absolute change in probability is small, so meaningful updates to rare tokens are restricted. DAPO introduces asymmetric clipping bounds to partially address this issue, but the constraint remains non-adaptive with respect to token probability.

Recent studies such as Yue et al. (2025) also suggest that the reasoning paths exploited by RLVR are largely present in the base model’s sampling distribution. This view is consistent with the small parameter shift observed during off-policy batch updates, reflected by the limited fraction of clipped parameters. Given this stability, we replace the fixed clipping rule in Equation (20) with a dynamic-adaptive restriction that includes the probability term directly, as in Equation (21).

$$\left| \left(\frac{p(x)}{q(x)} - 1 \right) p(x) \right| \leq \epsilon \quad (21)$$

Substituting $p(x) = r(x)q(x)$ into the preceding inequality and expanding the expression yields Equation (22).

$$-\epsilon \leq (r(x) - 1)r(x) * q(x) \leq \epsilon \quad (22)$$

For the clipping ratio to be well-defined, the underlying probabilities and ratio must satisfy the standard constraints in Equation (23). Specifically, $p(x)$ and $q(x)$ are probabilities in $[0, 1]$, the ratio $r(x)$ is non-negative, and the clipping hyperparameter ϵ is non-negative.

$$\begin{cases} 0 \leq p(x) \leq 1 \\ 0 \leq q(x) \leq 1 \\ 0 \leq r(x) \\ 0 \leq \epsilon \end{cases} \quad (23)$$

The admissible bound components derived from Equation (22) are defined in Equation (24), with conditions that ensure the resulting clipping interval is valid and practically usable.

$$\begin{cases} low_{ge}(q(x), \epsilon) = 0.5 - \frac{1}{2}\sqrt{1 + \frac{4\epsilon}{q(x)}} \\ high_{le}(q(x), \epsilon) = 0.5 + \frac{1}{2}\sqrt{1 + \frac{4\epsilon}{q(x)}} \\ low_{le}(q(x), \epsilon) = 0.5 - \frac{1}{2}\sqrt{1 - \frac{4\epsilon}{q(x)}}, & \text{S.t. } q(x)^2 - 4q(x) * \epsilon \geq 0 \\ high_{ge}(q(x), \epsilon) = 0.5 + \frac{1}{2}\sqrt{1 - \frac{4\epsilon}{q(x)}}, & \text{S.t. } q(x)^2 - 4q(x) * \epsilon \geq 0 \end{cases} \quad (24)$$

Combining Equation (22) with the bound definitions in Equation (24), the probability ratio $r(x)$ must satisfy the following conditions:

$$\begin{cases} low_{ge}(q(x), \epsilon) \leq 0 \leq r(x) \leq high_{le}(q(x), \epsilon), & \text{when } \text{Var}_{x \sim q} \left[f(x) \frac{p(x)}{q(x)} \right] \geq \text{Var}_{x \sim p} [f(x)] \\ r(x) \leq low_{le}(q(x), \epsilon) \cup high_{ge}(q(x), \epsilon) \leq r(x), & \text{otherwise} \end{cases} \quad (25)$$

To construct the clipping interval, we use the overlap region centered around ratio value 1. This choice minimizes the discrepancy between the two cases while preserving consistency with the conventional GRPO fixed bound, leading to the validity condition below:

$$\begin{aligned} high_{ge}(q(x), \epsilon) \leq r(x) \leq high_{le}(q(x), \epsilon) \\ \text{S.t. } q(x)^2 - 4q(x) * \epsilon \geq 0 \end{aligned} \quad (26)$$

For low_{le} and $high_{ge}$, the square-root argument must be non-negative. We therefore replace the argument with its maximum against zero, e.g., $\max(1 - \frac{4\epsilon}{q(x)}, 0)$. This yields the following **dynamic-adaptive clipping bounds for importance sampling**:

$$0.5 + \frac{1}{2}\sqrt{\max\left(1 - \frac{4\epsilon_{low}}{q(x)}, 0\right)} \leq r(x) \leq 0.5 + \frac{1}{2}\sqrt{1 + \frac{4\epsilon_{high}}{q(x)}} \quad (27)$$

For the upper and lower bounds in Inequality 27, we use separate hyperparameters ϵ_{high} and ϵ_{low} so that the adaptive bound aligns with the GRPO clipping interval at selected reference points.

A.6 ANALYZING THE DIFFERENCES BETWEEN DYNAMIC-ADAPTIVE CLIPPING AND FIXED CLIPPING

To align the dynamic-adaptive clipping bounds with the fixed symmetric clipping bounds at the boundary points $(\frac{1}{1+\epsilon}, 1)$ and $(1, 1 - \epsilon)$, as shown in Figure 4d, we solve the equations in Equation (28) for ϵ_{low} and ϵ_{high} .

$$\begin{cases} q(x) = \frac{1}{1+\epsilon} \\ 1 = 0.5 + \frac{1}{2} \sqrt{1 + \frac{4\epsilon_{high}}{q(x)}} \end{cases}
\begin{cases} q(x) = 1 \\ 1 - \epsilon = 0.5 + \frac{1}{2} \sqrt{\max(1 - \frac{4\epsilon_{low}}{q(x)}, 0)} \end{cases} \quad (28)$$

In common settings such as GRPO, the fixed clipping threshold ϵ is 0.2. Under the alignment equations, the corresponding adaptive hyperparameters are approximately $\epsilon_{low} = 0.16$ and $\epsilon_{high} = 0.2$. Here, $\epsilon_{low} \neq \epsilon_{high}$ serves to match the probability-dependent adaptive curve to fixed-bound reference points, which differs from DAPO’s asymmetric fixed clipping design.

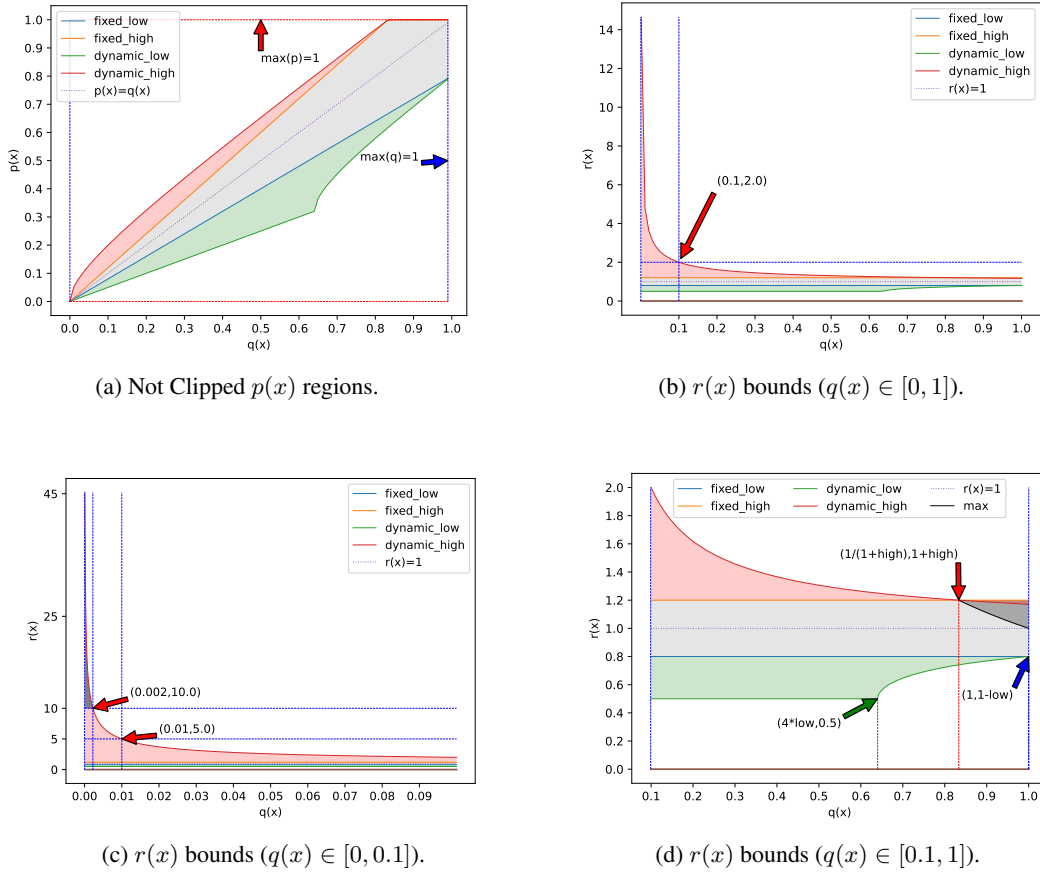


Figure 4: Comparison of fixed clipping bounds ($\epsilon = 0.2$) and dynamic-adaptive clipping bounds ($\epsilon_{low} = 0.16, \epsilon_{high} = 0.2$).

Dynamic versus fixed clipping: Figure 4 compares the fixed GRPO interval ($[1 - \epsilon, 1 + \epsilon]$) with the proposed probability-aware adaptive interval. For a token with old probability $q(x)$, fixed clipping provides constant relative width 2ϵ . As $q(x)$ decreases, the corresponding absolute exploration space contracts linearly (Figure 4a). By contrast, the relative width of the dynamic-adaptive interval scales inversely with $q(x)$, so the effective exploration space remains approximately stable across the probability range.

Effect of decreasing $q(x)$: When $q(x)$ becomes small, the admissible interval $[1 - \epsilon, 1 + \epsilon]$ under fixed clipping is unchanged, so the model receives no additional capacity to update low-probability tokens. The dynamic-adaptive bound instead expands rapidly (Figure 4b), creating a larger relative update region for rare tokens, which is the desired behavior for efficient exploration in RL.

Behavior for extremely low probabilities: Figure 4c shows the clipping interval for $q(x) \in [0, 0.1]$. As $q(x) \rightarrow 0$, the dynamic-adaptive upper bound for $r(x)$ grows rapidly with inverse square-root dependence, whereas the fixed-clipping upper bound remains constant. Following the dual-clipping strategy (Ye et al., 2020), we impose a hard ceiling $r_{\max} = 10$ to avoid numerical and optimization instabilities such as gradient explosion or vanishing. The gray region in Figure 4c corresponds approximately to $q(x) \leq 2 \times 10^{-3}$, where the adaptive upper bound is capped at $r_{\max}$.

Upper and lower bounds: For the lower bound, when $q(x) \leq 4\varepsilon$ the dynamic-adaptive rule gives $dynamic_{low} = 0.5$, whereas fixed clipping keeps $fixed_{low} = 1 - \varepsilon = 0.8$. As $q(x)$ increases beyond this threshold, the dynamic-adaptive lower bound smoothly approaches the fixed value. Similarly, the dynamic-adaptive upper bound exceeds the fixed bound when $q(x) < \frac{1}{1+\varepsilon_{high}}$ and coincides with it for larger $q(x)$. Thus, the adaptive interval $[dynamic_{low}(q(x), \epsilon_{low}), dynamic_{high}(q(x), \epsilon_{high})]$ widens the admissible region for rare tokens while retaining the tight fixed interval for common tokens (Figure 4d).

Positive versus negative advantage: The loss in Equation (8) includes cases with positive and negative advantages. Under the minimum operation, the ratio $r(x)$ for a positive advantage is clipped in $[0, dynamic_{high}(q(x), \epsilon_{high})]$, whereas for a negative advantage the clipping interval is $[dynamic_{low}(q(x), \epsilon_{low}), \infty)$. Because excessively large $r(x)$ can destabilize training, we cap the ratio for both positive and negative advantages at $r_{\max} = 10$. This safeguard follows Ye et al. (2020) and prevents the adaptive interval from generating excessively large importance weights.

In summary, for both positive and negative advantages, dynamic-adaptive clipping provides a larger admissible update region than fixed clipping for rare low-probability tokens, thereby increasing the model’s capacity for diversity exploration.

A.7 THE DEFINITIONS OF THE ADVANTAGE STANDARDIZATION

Let i denote the i -th iteration in which prompts are used to generate responses and update model parameters. We define the following reward-normalization statistics.

The response-reward means are defined by the functions in Equation (29):

$$\begin{cases} \mu_{new}^i = \frac{1}{G} \sum_{j=1}^G R_j^i \\ \mu_{old}^i = \frac{1}{G*(i-1)} \sum_{k=1}^{i-1} \sum_{j=1}^G R_j^k \\ \mu_{total}^i = \frac{1}{G*i} \sum_{k=1}^i \sum_{j=1}^G R_j^k \\ \quad = \frac{1}{i} (\mu_{new}^i + (i-1) \mu_{old}^i) \end{cases} \quad (29)$$

The corresponding response-reward standard deviations are defined in Equation (30):

$$\begin{cases} \sigma_{new}^i = \sqrt{\frac{1}{G} \sum_{j=1}^G (R_j^i - \mu_{new}^i)^2} \\ \sigma_{old}^i = \sqrt{\frac{1}{G*(i-1)} \sum_{k=1}^{i-1} \sum_{j=1}^G (R_j^k - \mu_{old}^i)^2} \\ \sigma_{total}^i = \sqrt{\frac{1}{G*i} \sum_{k=1}^i \sum_{j=1}^G (R_j^k - \mu_{total}^i)^2} \\ \quad = \sqrt{\frac{1}{i} \left(\sigma_{new}^i{}^2 + (i-1) \sigma_{old}^i{}^2 + \frac{i-1}{i} (\mu_{old}^i - \mu_{new}^i)^2 \right)} \end{cases} \quad (30)$$

To reduce computational and storage overhead when computing σ_{total}^i and μ_{total}^i in Equation (30) and Equation (29), we use equivalent expressions based on μ_{new}^i , σ_{new}^i , μ_{old}^i , and σ_{old}^i rather than recomputing statistics from all original rewards.

A.8 THE TEMPLATE FOR EXPERIMENTS

All experiments use the following prompt template, adapted from the Qwen-Math template (Yang et al., 2024b).

Qwen-Math Template

```
<|im_start|>system
Please reason step by step, and put your final answer within \boxed{ }. <|im_end|>
<|im_start|>user
{problem}
<|im_end|>
<|im_start|>assistant
```

A.9 TRAINING SETTING

For GRPO, we set the symmetric clipping threshold to $\epsilon = 0.2$. For DAPO, we use lower/upper clipping thresholds (0.2, 0.28), a 512-token soft-punish cache, and maximum probability ratio 10 for negative advantages. For GSPO, we use lower/upper clipping thresholds $(3e - 4, 4e - 4)$. For DCPO, we use low/high clipping thresholds (0.16, 0.2), which preserve the GRPO clipping reference points $(q(x), r(x)) = (\frac{1}{1+\epsilon}, 1 + \epsilon)$ and $(q(x), r(x)) = (1, 1 - \epsilon)$ shown in Figure 4d; unlike DAPO, DCPO sets the maximum probability ratio to 10 for both positive and negative advantages.

Training is conducted for 400 steps with generation batch size 512 and parameter-update mini-batch size 32. For each problem, we generate $G=16$ responses with temperature 1 and top_p 1. The maximum prompt length is 1,024 tokens and the maximum response length is 3,072 tokens, which is substantially shorter than the 20k maximum length used in the original DAPO setting.

The training implementation is adapted from Verl (Sheng et al., 2024) and executed on 32xH20 GPUs. Unless otherwise specified, hyperparameters follow DAPO. For loss computation, we retain the original $\frac{1}{G}$ or $\sum_{i=1}^G \frac{1}{|o_i|}$ weighted-average formulation from GRPO or DAPO rather than the approximate weighted calculation used in Verl, ensuring compatibility across different microbatch sizes.

A.10 THE DETAILED PERFORMANCE OF MODELS

Avg@1 Trend Figure 5 reports average Avg@1, computed by averaging greedy-decoding accuracy over MATH500, AMC23, AIME24, and AIME25. This metric measures deterministic best-response performance. DCPO consistently outperforms GRPO and DAPO across all four models and outperforms GSPO on the two models for which GSPO is evaluated. Its trajectory is also more stable than the baselines, avoiding the pronounced performance drop observed for GRPO and the large fluctuations observed for DAPO. These results indicate that DCPO improves deterministic reasoning performance while maintaining stable optimization dynamics.

Avg@32 Trend Figure 6 compares Avg@32 performance for DCPO, GRPO, DAPO, and, where available, GSPO across model sizes on AMC23, AIME24, and AIME25. The curves show that DCPO provides stronger robustness and stability under stochastic decoding. GRPO exhibits substantial fluctuations and occasional performance collapse; for Qwen2.5-14B, GRPO shows a marked performance drop after approximately 170 training steps. Across all evaluated model sizes, from 1.5B to 14B, DCPO follows a consistently increasing trajectory with limited fluctuation. DCPO also improves rapidly in early training and maintains the upward trend thereafter. Relative to DAPO, DCPO achieves higher performance while using approximately half the training time and resources.



Figure 5: Avg@1 performance averaged across benchmarks.

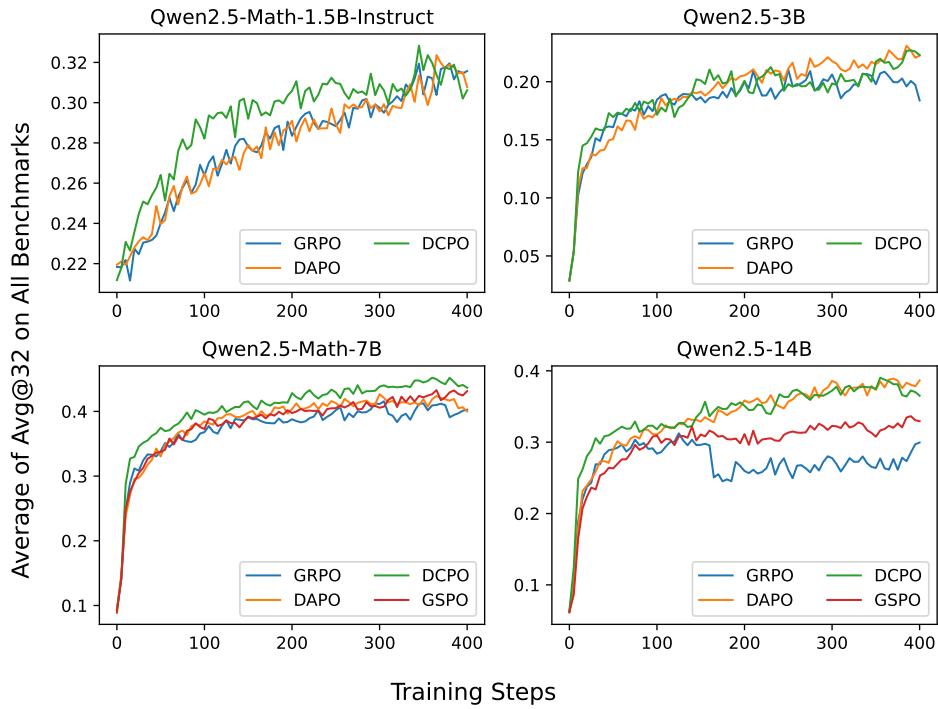


Figure 6: Avg@32 performance averaged across benchmarks.

These results demonstrate that dynamic-adaptive clipping and smoothed advantage standardization improve exploration at both response and label-distribution levels.

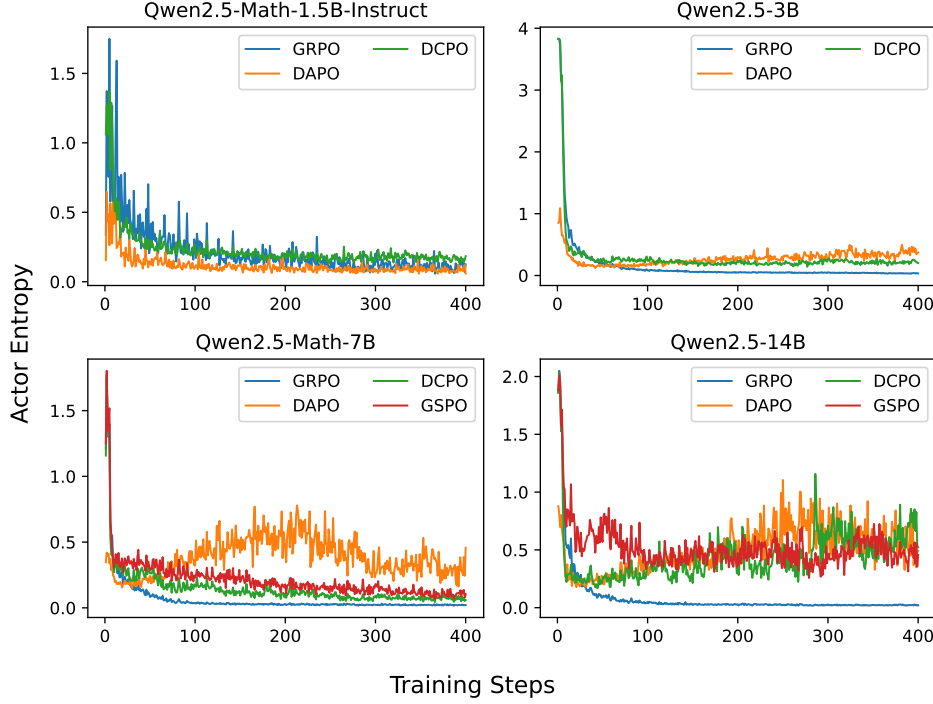


Figure 7: Model entropy during training.

Entropy Trend Figure 7 shows that GRPO undergoes a sharp entropy decrease after an initial warm-up phase and then stabilizes in a low-entropy regime. This entropy collapse indicates rapid loss of policy randomness, which reduces sampled-response diversity and constrains subsequent policy optimization.

The entropy behavior of DAPO and DCPO depends on model type. For the Instruct model (Qwen2.5-Math-1.5B), DAPO, DCPO, and GRPO follow similar decreasing trajectories. For the larger base models (Qwen2.5-3B, Qwen2.5-Math-7B, and Qwen2.5-14B), the methods diverge. DAPO maintains the highest entropy and exhibits an upward trend with large fluctuations. On Qwen2.5-Math-7B, GSPO entropy is similar to DCPO entropy, and the average performance metrics are also comparable. On Qwen2.5-14B, however, GSPO performance follows a fluctuating downward trend and yields weaker results. DCPO instead maintains a moderate entropy level: its curve is smoother and remains within a bounded band between the low-entropy GRPO regime and the high-variance DAPO regime.

These observations suggest a U-shaped relationship between entropy and training performance. Excessively low entropy removes exploratory signal and can cause premature convergence, whereas excessively high or unstable entropy degrades optimization stability. DCPO maintains entropy in a moderate range, supporting both convergence and sufficient exploration; this effect becomes more pronounced as model capacity increases. The entropy profile therefore provides a plausible mechanism for DCPO’s stronger inference performance and more robust policy learning across model scales.